FastAPI Layer — apm_connectors/api/

A from-zero Python syntax primer, a from-zero FastAPI tutorial, then a complete deep dive into this repo's API layer and the design patterns it uses — written as a standalone reference, including for interview prep.

Part 0 — Python syntax you'll see throughout this document

If you're new to Python, read this part first — every construct here reappears constantly in Parts 1-4. Each one is explained with a plain toy example, then the actual line from this codebase that uses it.

0.1 Imports — how one file uses code from another

```
from fastapi import FastAPI          # "from the fastapi package, bring in the name FastAPI"
import os                            # "bring in the whole os module; refer to it as
    os.something"
from apm_connectors.graph import resume_process, start_action
```

A *module* is just one `.py` file; a *package* is a folder of modules (like `apm_connectors.api`, the folder this whole document is about). `from X import Y` pulls one specific name — a function, a class, a variable — out of module/package X so you can use it directly by that name, instead of writing `X.Y` every time.

0.2 Functions — parameters, default values, return type hints

```
def greet(name, excited=False):        # `excited` has a default -- calling greet("Sam") is fine
    if excited:
        return name + "!"
    return name

greet("Sam")             # "Sam"   -- excited defaults to False
greet("Sam", True)       # "Sam!"
greet(name="Sam")        # same as the first call, but naming the argument explicitly
```

Any parameter with `= something` after it becomes optional — the caller may omit it, and the default is used. Calling with `name="Sam"` instead of just `"Sam"` is a *keyword argument* — same effect, more explicit at the call site. This repo's route functions do this constantly: `def list_items(limit: int = 10):` means `limit` is optional and defaults to 10.

0.3 Type hints — annotations that tools read, but plain Python doesn't enforce

Writing `x: int` instead of just x *documents* that x should be an integer — bare Python itself does not stop you from passing a string anyway. What makes type hints load-bearing in this codebase specifically is that **Pydantic and FastAPI read them at runtime** and actively validate against them (§1.4) — the hint stops being just a comment and becomes an enforced rule, but only because a library chose to enforce it, not because Python itself does.

```
age: int                    # a plain int
name: str | None = None     # either a str, or None -- and optional, since it has a default
tags: list[str]             # a list containing only strings
tools: dict[str, BaseTool]  # a dict whose keys are str and whose values are BaseTool instances
```

`X | None` (Python 3.10+) is the modern way to write "this is either type X, or the value None" — you may also see the older, equivalent `Optional[X]` from the `typing` module in other codebases; they mean the same thing. `list[str]` and `dict[str, X]` are *generic* type hints — they say not just "a list" but "a list of specifically these". You'll see this exact shape everywhere in schemas.py: `process_id: str | None = None` —

optional, defaults to None if the caller doesn't send one.

0.4 Classes — self, inheritance, and attribute declarations

```
class Animal:
    def __init__(self, name):     # runs automatically when you write Animal("Rex")
        self.name = name          # `self` = "this particular instance"
    def speak(self):
        return self.name + " makes a sound"

class Dog(Animal):                # Dog inherits everything Animal has, then can add/override
    def speak(self):
        return self.name + " barks"

Dog("Rex").speak()   # "Rex barks"
```

`self` is just a name (by convention, always the first parameter of a method) referring to the specific instance the method was called on — Python passes it in automatically whenever you write `instance.method()`; you never pass it yourself. `class Dog(Animal):` means Dog *inherits* from Animal — it starts with everything Animal has, and can add new behavior or override existing methods. Pydantic's `BaseModel` (§1.4) uses this exact mechanism: every request schema in this repo is `class SomeRequest(BaseModel):`, inheriting all of Pydantic's validation behavior for free.

One more thing worth flagging because it looks unusual the first time: inside a `BaseModel` subclass, a line like `name: str` with no `self.` and no function body is *not* a regular Python statement — it's Pydantic-specific class syntax declaring "this model has a required field called name, of type str". You won't see this pattern outside of Pydantic models (and the similar standard-library `@dataclass`).

0.5 Decorators — what @something above a function actually does

A decorator is a function that takes your function as input and gives back a (possibly different) function. Writing `@decorator` directly above a `def` is shorthand for calling the decorator on it afterward:

```
@app.get("/health")
def health():
    ...

# is (roughly) shorthand for:

def health():
    ...
health = app.get("/health")(health)     # app.get(...) returns a decorator, which wraps health
```

This is how a library "registers" or "wraps" your function without you writing an extra line of registration code yourself. `@app.get("/health")` registers the function as a route handler (§1.2); `@lru_cache` (§2.3) wraps a function so repeated calls with the same arguments return a cached result instead of recomputing. Same mechanism both times — only what the decorator *does* to your function differs.

0.6 f-strings — building a string with values inside it

```
name = "Sam"
greeting = f"Hello, {name}!"     # "Hello, Sam!" -- whatever's inside {} is evaluated and
    inserted
```

The `f` right before the opening quote is what activates this — without it, `{name}` would just be four literal characters, not a substitution. This repo builds human-readable audit descriptions this way constantly: `f"Create Salesforce {body.object_name} record"`.

0.7 Dicts, lists, and comprehensions

```
{"to": to, "subject": subject}          # a dict literal -- key: value pairs
[r.__dict__ for r in results]           # a list comprehension

# the comprehension above is shorthand for:
output = []
for r in results:
    output.append(r.__dict__)
```

A *list comprehension* — `[expression for item in iterable]` — builds a new list in one line instead of a multi-line for-loop with `.append(...)`; every read route in this repo ends with exactly this shape to convert a list of connector objects into plain JSON-able dicts. `r.__dict__` is worth knowing specifically: every ordinary Python object keeps its attributes internally as a dict named `__dict__` — accessing it directly is a quick way to turn an object into a plain `dict` FastAPI can serialize to JSON.

0.8 Exception handling — try/except, and raise ... from ...

```
try:
    risky_call()
except ValueError as exc:      # only catches ValueError -- others propagate up
    handle(exc)

try:
    results = tool.query_records(process_id, soql=body.soql)
except Exception as exc:
    raise upstream_error(exc) from exc    # wraps exc, keeping it attached
```

`except Exception as exc:` catches the error and binds it to the name `exc` so you can inspect or reuse it. `raise NewError(...) from exc` is Python's *exception chaining*: it raises a new, more specific exception (in this repo, always an `HTTPException` — §1.7) while keeping the original exception attached as context, so a server-side traceback shows both "here's the clean error the caller saw" and "here's what actually broke underneath it", instead of losing the original cause.

0.9 async def and await — a quick pointer

You'll see `async def` instead of plain `def` on some functions, and `await` before some calls inside them. This is covered properly in §1.6 once you've seen how FastAPI runs — the short version for now: `async def` marks a function as one that can pause and let other work happen while it waits on something slow (a network call), and `await` marks exactly where it pauses.

0.10 A local import inside a function body

```
def get_state_store():
    if settings.database_url:
        from apm_connectors.state.postgres_store import PostgresStateStore   # imported HERE,
            not at the top of the file
        return PostgresStateStore(...)
    return StateStore()
```

Imports usually live at the very top of a file, but here's one deliberately placed inside the function instead. This delays loading the Postgres driver until this exact line actually runs — which only happens if `DATABASE_URL` is set. Without this, importing `apm_connectors.api.dependencies` at all would require the Postgres driver to be installed, even for a deployment that never sets `DATABASE_URL` and only wants the zero-infrastructure file-backed default (§2.3). You'll see this same local-import pattern again for the same reason.

Part 1 — FastAPI from zero

1.1 What FastAPI actually is

FastAPI is a Python web framework for building HTTP APIs. Three facts explain almost everything else about it:

It's built on Starlette (the actual ASGI toolkit that handles routing, requests, responses, middleware) **and Pydantic** (the data-validation library). FastAPI itself is mostly glue: it reads your Python type hints and wires them into Starlette's routing and Pydantic's validation automatically. **It's ASGI, not WSGI** — ASGI (Asynchronous Server Gateway Interface) is the newer standard that supports async request handling, WebSockets, and long-lived connections; WSGI (what Flask/Django classically used) is synchronous, one thread per request. **It generates its own API documentation** from your code — every route's parameters, types, and response shape are introspected at startup and served live at `/docs` (Swagger UI) and `/redoc`, with zero extra work from you.

1.2 Path operations — the core building block

A **path operation** is a Python function decorated to handle one HTTP method on one URL path:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

`@app.get` registers this function against `GET /health`; there's a matching `@app.post`, `@app.put`, `@app.delete`, etc. Whatever you `return` — a dict, a list, a Pydantic model — FastAPI serializes to JSON automatically. This decorator pattern is the entire routing mechanism: there's no separate URL-config file to maintain, the route lives right next to the function that handles it.

1.3 Getting data in: path params, query params, request body

Three ways a caller sends FastAPI data in, distinguished purely by where you put the parameter:

```
@app.get("/items/{item_id}")           # path parameter -- part of the URL itself
def get_item(item_id: int):              # GET /items/42  ->  item_id = 42
    ...

@app.get("/items")                      # query parameter -- a plain, non-body function arg
def list_items(limit: int = 10):          # GET /items?limit=5  ->  limit = 5
    ...

class Item(BaseModel):                    # request body -- a Pydantic model as the arg type
    name: str
    price: float

@app.post("/items")                     # POST /items, JSON body {"name":..., "price":...}
def create_item(item: Item):
    ...
```

FastAPI decides which of these a parameter is purely from *how it's declared*: named in the URL path template → path parameter; a plain scalar type not in the path → query parameter; a Pydantic `BaseModel` subclass → request body, parsed from JSON. No separate parsing code anywhere — this repo's routes take the request-body form exclusively, covered in Part 2.

1.4 Pydantic models: type hints that validate themselves

A `BaseModel` subclass isn't just a type-hint convenience — FastAPI uses it to validate every incoming request body before your function body ever runs:

```
class GmailSendRequest(BaseModel):
    to: str
    subject: str
    body: str
    process_id: str | None = None    # optional -- a default makes a field non-required
```

POST a body missing `subject`, or with `to` as a number instead of a string, and FastAPI returns a `422 Unprocessable Entity` with a field-by-field error list — *before* your route function is called at all. You never write `if "subject" not in body: raise ...` — the type hints *are* the validation rules.

1.5 Dependency Injection — the concept that makes FastAPI's design different

This is the single most distinctive FastAPI idea, and the one most worth understanding properly (this repo leans on it heavily — see §2.3). The problem: a route often needs some shared object — a database connection, a config, an authenticated user — and you don't want to construct it by hand inside every single route function. **Depends()** solves this: declare a parameter's default as `Depends(some_function)`, and FastAPI calls `some_function()` for you before your route runs, then passes its return value in as that parameter.

```
def get_db():
    return Database(...)          # could open a connection, read config, anything

@app.get("/users/{id}")
def get_user(id: int, db: Database = Depends(get_db)):
    return db.query(...)          # `db` arrived already built -- this function never called
        get_db() itself
```

This is real dependency injection, not just a naming convention: the route function declares *what it needs* (a `Database`), and something outside the function decides *how to build it*. Swap what `get_db` returns — a real database in production, a fake one in tests — and every route using it changes behavior with zero edits to the routes themselves. FastAPI also builds a dependency *graph*: a dependency can itself take a `Depends(...)` parameter, and FastAPI resolves the whole chain, caching each dependency's result once per request by default so calling the same dependency twice in one request doesn't rebuild it twice.

1.6 async def vs def — when it actually matters

FastAPI lets a path operation be either `async def` or plain `def`, and both work — the difference is about the server's event loop, not correctness. An ASGI server (see §1.8) runs one event loop handling many requests concurrently by *interleaving* them whenever a request hits an `await` (e.g. an async database call, an async HTTP request). If a route is `async def` but calls something *blocking* inside it (a synchronous network call, a heavy CPU loop) without awaiting it, that call blocks the *entire event loop* — every other in-flight request stalls too, not just this one. A plain `def` route is automatically run by FastAPI in a separate thread pool instead, so a blocking call inside it only blocks that one thread. Rule of thumb: `async def` only if everything inside genuinely uses `await`; otherwise plain `def` is the safer default.

1.7 Response models, status codes, and error handling

```
@app.post("/items", response_model=ItemOut, status_code=201)
def create_item(item: Item) -> ItemOut:
    ...

from fastapi import HTTPException

@app.get("/items/{id}")
def get_item(id: int):
    if id not in db:
        raise HTTPException(status_code=404, detail="not found")
    return db[id]
```

`response_model` does two things: it filters the returned object down to exactly that shape (extra attributes on what you return are silently dropped, not leaked to the caller), and it's what populates the auto-generated docs' "response schema" section. `HTTPException` is how a route signals a clean, intentional error — raise it anywhere in a route (or in a dependency) and FastAPI turns it into a proper JSON error response with that status code, short-circuiting the rest of the function.

1.8 How it actually runs: ASGI servers and Uvicorn

FastAPI itself is just an application object — something needs to actually accept TCP connections, parse HTTP, and call into it. That's an **ASGI server**, almost always **Uvicorn** in practice:

```
uvicorn apm_connectors.api.app:app --reload --port 8000
```

`apm_connectors.api.app:app` means "import the module `apm_connectors.api.app` and use its module-level variable named `app`" — that variable is the actual FastAPI instance Uvicorn drives. `--reload` watches source files and restarts on change (dev only). This detail matters for understanding this repo's `app.py` in Part 2, which ends with exactly that module-level `app = create_app()` line.

Part 2 — This repo's FastAPI layer, file by file

src/apm_connectors/api/ has five files. Read them in this order — each one only makes sense once you've seen the one before it:

File	Lines	Role
schemas.py	148	Pydantic request/response models — the request-body shape for every route.
dependencies.py	133	Builds the shared tools/state-store/action-graph once per process; the Depends() targets.
_responses.py	30	Two tiny shared helpers: error translation, and RunOutcome → response conversion.
tools_routes.py	319	The actual /tools/* routes — one read + one write route pair per connector.
app.py	57	Builds the FastAPI app, mounts tools_routes' router, adds /health and /processes/*.

2.1 schemas.py — the request/response contract

One `BaseModel` per route, one field per keyword argument the matching connector method takes, plus an always-optional `process_id`:

```
class GmailSendRequest(BaseModel):
    process_id: str | None = None
    to: str
    subject: str
    body: str

class RunOutcomeResponse(BaseModel):
    """Mirrors apm_connectors.graph.RunOutcome. Exactly one of
    pending_action / final_result is set."""
    action_id: str
    summary: str | None
    pending_action: dict[str, Any] | None
    final_result: dict[str, Any] | None
```

Two design choices worth naming. First, **process_id is optional everywhere** — a calling reasoning layer won't always have an internal case id, and shouldn't need to invent one just to call a connector; the server generates one internally when omitted (§2.4). Second, **RunOutcomeResponse is a discriminated-by-convention union**: exactly one of `pending_action`/`final_result` is ever non-null, and every write route shares this one response shape — a caller checks which field is set rather than branching on route-specific response types.

2.2 _responses.py — the error-translation seam

```
def upstream_error(exc: Exception) -> HTTPException:
    """Turn an unexpected failure ... into a clean 502 response with a
    readable message, instead of letting an unhandled 500 with a raw
    Python traceback reach the caller."""
    return HTTPException(status_code=502, detail=f"Upstream tool error: {exc}")

def to_response(outcome: RunOutcome) -> RunOutcomeResponse:
    return RunOutcomeResponse(
        action_id=outcome.process_id, summary=outcome.summary,
        pending_action=outcome.pending_action, final_result=outcome.final_result,
    )
```

Two tiny functions, but they exist specifically so `app.py` and `tools_routes.py` never need to import from each other — both import this shared, dependency-free module instead. `upstream_error` is the one place a raw connector-level exception (a network error, an exhausted retry) becomes a clean API error; every route wraps its risky call in `try/except Exception as exc: raise upstream_error(exc) from exc`.

2.3 dependencies.py — the Depends() targets, and the Postgres toggle

Three functions, each decorated `@lru_cache` — every route's `Depends(get_tools)`/`Depends(get_action_graph)` points here:

```
@lru_cache
def get_state_store() -> StateStoreProtocol:
    settings = load_settings()
    if settings.database_url:
        from apm_connectors.state.postgres_store import PostgresStateStore
        return PostgresStateStore(_get_postgres_pool())
    return StateStore()

@lru_cache
def get_tools() -> dict[str, BaseTool]:
    ...  # builds Gmail/Calendar/Excel/Salesforce/Jira, whichever have credentials configured

@lru_cache
def get_action_graph():
    ...  # MemorySaver checkpointer, or PostgresSaver if settings.database_url is set
    return build_action_graph(get_tools(), get_state_store(), checkpointer=checkpointer)
```

Why @lru_cache and not a fresh object per request. Depends() by default resolves a fresh call *per request* — but the compiled action graph's checkpointer holds paused, mid-approval state *between* requests (a write proposed on one request is approved on a later, separate request). A per-request `get_action_graph()` would silently lose that state between the two calls. `@lru_cache` on a zero-argument function makes it a process-wide singleton instead: built once, on first use, reused for the life of the process. This is also exactly why the tool set, the state store, and the compiled graph must all be built through these cached functions and never constructed directly inside a route.

The DATABASE_URL branch is a Strategy pattern, chosen once at startup: the same `Depends(get_state_store)` call site works identically whether it resolves to a JSON-file store or a Postgres-backed one — nothing downstream branches on which. See the State Store and Persistence sections of the architecture reference doc for the full story on why this exists.

2.4 tools_routes.py — the actual routes, and the two shapes

Every one of the eighteen routes here is one of exactly two shapes. A **read** calls the tool directly and returns data immediately:

```
@router.post("/salesforce/query")
def salesforce_query(body: SalesforceQueryRequest, tools: dict[str, BaseTool] =
    Depends(get_tools)) -> list[dict]:
    tool = _tool(tools, "salesforce")
    process_id = _resolve_process_id(body.process_id)
    try:
        results = tool.query_records(process_id, soql=body.soql)
    except Exception as exc:
        raise upstream_error(exc) from exc
    return [r.__dict__ for r in results]
```

A **write** never calls the tool's write method at all — it hands the request to the action graph instead, via the shared `_propose` helper:

```
def _propose(graph, process_id, tool, method, description, payload) -> RunOutcomeResponse:
    try:
        outcome = start_action(graph, process_id, tool=tool, method=method,
                               description=description, payload=payload)
    except Exception as exc:
        raise upstream_error(exc) from exc
    return to_response(outcome)

@router.post("/salesforce/create", response_model=RunOutcomeResponse)
def salesforce_create(body: SalesforceCreateRequest, tools=Depends(get_tools),
    graph=Depends(get_action_graph)):
    _tool(tools, "salesforce")  # fail fast, before recording a pending action doomed to fail
    description = f"Create Salesforce {body.object_name} record"
    payload = {"object_name": body.object_name, "fields": body.fields}
    return _propose(graph, action_id, "salesforce", "create_record", description, payload)
```

Three tiny module-level helpers make every one of the eighteen routes a 3-6 line function: `_tool(tools, name)` raises a clean 503 if that connector isn't configured on this server (rather than the route breaking with an unrelated error); `_resolve_process_id(process_id)` generates a `uuid4()` when the caller omitted one; `_propose(...)` is the one call every write route makes into the action graph. And there's exactly **one** decision route, shared by every write, because approving or rejecting is the same operation no matter which connector proposed the action:

```
@router.post("/actions/{action_id}/decision", response_model=RunOutcomeResponse)
def decide_action(action_id: str, body: DecisionRequest, graph=Depends(get_action_graph)) ->
    RunOutcomeResponse:
    try:
        outcome = resume_process(graph, action_id, approved=body.approved)
    except Exception as exc:
        raise upstream_error(exc) from exc
    return to_response(outcome)
```

Note the `APIRouter` at the top of the file — `router = APIRouter(prefix="/tools", tags=["tools"])` — every route in this file is declared against `router`, not `app` directly. That's what lets `app.py` mount this whole file's worth of routes in one line (§2.5): `APIRouter` is FastAPI's way of splitting a large app's routes across files/modules without every route needing a reference to the top-level app object.

2.5 app.py — assembling the app

```
def create_app() -> FastAPI:
    app = FastAPI(title="APM Connectors & Enterprise Systems API")
    app.include_router(tools_router)          # every /tools/* route, mounted in one line

    @app.get("/health")
    def health() -> dict[str, str]:
        return {"status": "ok"}

    @app.get("/processes/{process_id}/status")
    def get_process_status(process_id: str, store: StateStore = Depends(get_state_store)) ->
        dict:
        status = store.get_status(process_id)
        if status is None:
            raise HTTPException(status_code=404, detail=f"unknown process_id: {process_id}")
        return status

    # ... /processes, /processes/{id}/history, /processes/{id}/pending, same shape ...
    return app

app = create_app()   # module-level instance -- what uvicorn actually imports and runs
```

Why a factory function (create_app()) instead of just a module-level FastAPI() call. A function that *builds and returns* an app can be called more than once, with different results each time — critically, this is what lets tests build a fresh app per test case rather than sharing one global mutable instance across an entire test run (see §2.8). The module-level `app = create_app()` at the bottom exists purely for Uvicorn, which needs a real importable variable to point at (§1.8) — it's not what tests use.

Part 3 — Design patterns used here, named explicitly

Interview-relevant framing: these aren't decorative labels, each one is doing real work in this codebase. Being able to name the pattern *and* point at the concrete problem it solves here is the difference between having read about a pattern and having used one.

Pattern	Where	What problem it actually solves here
Dependency Injection	Every route's `Depends(get_tools)` / `Depends(get_action_graph)`	Routes declare what they need, not how to build it — swapping real tools for fake ones in tests needs zero route-code changes (§2.8).
Singleton (via @lru_cache)	dependencies.py's three cached functions	The compiled action graph's checkpointer must persist paused state across separate requests — a fresh instance per request would lose it (§2.3).
Factory Function	create_app(), build_action_graph(...), build_server(...) (MCP layer)	Same shape everywhere in this codebase: take dependencies as parameters, return a built object, so tests inject different dependencies without subclassing anything.
Strategy (pluggable implementation)	get_state_store()'s DATABASE_URL branch	Two interchangeable persistence backends (file vs. Postgres) behind one call site — the caller never branches on which is active.
Adapter / Protocol	StateStoreProtocol (state/store.py)	Callers depend on a method surface, never a concrete class — this is what makes the file-store-to-Postgres swap possible without touching routes.
Facade	tools_routes.py's route functions	Each route is a thin 3-6 line facade over a much larger subsystem (the connector layer + the action graph) — callers never see that complexity.
Chain of Responsibility (error translation)	raw exception → HTTPException (here) → ConnectorAPIError (MCP client) → ToolError (MCP server)	Each layer boundary owns translating failures into its own vocabulary, so nothing downstream ever sees a raw traceback.
Router / Module pattern	APIRouter(prefix="/tools") + app.include_router(...)	Splits a large app's routes across files without every route needing direct access to the top-level FastAPI instance.

3.1 Testing pattern: dependency_overrides

FastAPI's dependency injection has a matching testing mechanism: `app.dependency_overrides` is a dict mapping a dependency function to a replacement, checked before FastAPI would normally call the real one:

```
app = create_app()
app.dependency_overrides[get_tools] = lambda: fake_tools_dict
app.dependency_overrides[get_action_graph] = lambda: fake_action_graph

client = TestClient(app)
response = client.post("/tools/salesforce/query", json={"soql": "SELECT Id FROM Account"})
```

This is the direct payoff of §2.3's design choice: because every route depends on `get_tools`/`get_action_graph` rather than importing concrete tool classes, a test can redirect *only those two functions* and every route automatically uses fake, no-live-credentials-needed connectors — no route code changes, no monkeypatching internals. `TestClient` itself runs requests in-process against the ASGI app object directly (no real socket, no separately-running Uvicorn) — the same in-process-ASGI idea the MCP server layer's own tests reuse via `httpx.ASGITransport`.

3.2 Full route table

Route	Kind	Connector method
POST /tools/gmail/search, /read	read	search_emails, read_message
POST /tools/gmail/send	write	send_email
POST /tools/calendar/search, /read	read	search_events, read_event
POST /tools/calendar/create-event	write	create_event
POST /tools/excel/worksheets, /read	read	list_worksheets, read_range
POST /tools/excel/write	write	write_range
POST /tools/salesforce/query, /read	read	query_records, get_record
POST /tools/salesforce/create, /update	write	create_record, update_record
POST /tools/jira/search, /read	read	search_issues, get_issue
POST /tools/jira/create, /update	write	create_issue, update_issue
POST /tools/actions/{action_id}/decision	shared decision	resume_process
GET /health, /processes, /processes/{id}/*	status/audit	StateStore reads

---

Part 4 — Interview-prep Q&A

Likely questions this codebase can answer concretely, not just in the abstract. Practice answering by pointing at the actual code, not reciting the definition.

Q: What is dependency injection, and why not just call the constructor directly in the route?

A: The route declares a parameter typed as Depends(some_function) instead of building the object itself. FastAPI resolves it before the route runs. The payoff is substitutability: tests override get_tools/get_action_graph to inject fakes (§3.1) with zero changes to route code — impossible if routes called GmailTool(...) directly.

Q: Why @lru_cache instead of a module-level global for the shared state store/tools/graph?

A: @lru_cache on a zero-arg function gives you a lazily-built singleton — built on first call, not at import time (so tests that override dependency_overrides before first use never trigger the real construction at all), versus a

module-level global that would build eagerly on import regardless of whether a test needs it.

Q: How does FastAPI know whether a parameter is a path param, query param, or body?

A: By where it's declared: named in the route's path template -> path param; a plain scalar not in the path -> query param; a Pydantic BaseModel subclass -> request body, parsed from JSON (§1.3).

Q: What actually happens on a 422 response?

A: Pydantic validation failed on the request body before the route function was even called -- a required field was missing, or a field's value didn't match its declared type. This repo's routes never hand-check for missing fields because Pydantic already guarantees they're present and correctly typed by the time the function body runs.

Q: async def or def -- how do you decide?

A: async def only if the function awaits everything inside it; otherwise plain def, because FastAPI runs sync routes in a thread pool automatically, so a blocking call there can't stall the whole event loop the way it would inside an async def that forgot to await it (§1.6).

Q: Why does this API have exactly one decision route instead of e.g. /tools/gmail/send/approve?

A: Approving or rejecting is identical regardless of which connector proposed the action -- it always resolves to the same resume_process(graph, action_id, approved) call. One shared route means one place to test and reason about the approval mechanic, instead of duplicating it once per connector.

Q: Where would you add a new connector's routes, and what would break if you changed an existing route's response shape?

A: Add new routes to tools_routes.py following the read/write shapes in §2.4 -- that's additive. Changing an existing route's request/response shape is a breaking change for any caller already coded against docs/api-contract.md -- CLAUDE.md calls this out explicitly as the line between safe and unsafe API changes.

Q: How is this tested without hitting real Gmail/Salesforce/Jira APIs?

A: dependency_overrides swaps get_tools/get_action_graph for versions built on fake HTTP clients, and TestClient drives requests through the real ASGI app in-process -- real route handlers, real Pydantic validation, real graph logic, zero live credentials or network calls (§3.1).

Vocabulary cheat-sheet

Term	Meaning here
`Type hint (x: int)`	Advisory annotation of expected type -- ignored by plain Python at runtime, but read and enforced by Pydantic/FastAPI (§0.3).
`X | None`	Union type meaning "X, or None" (Python 3.10+); older code writes the equivalent Optional[X] instead.
`Decorator (@x)`	A function that wraps another function, applied via @ syntax right above a def (§0.5) -- e.g. @app.get(...), @lru_cache.
`List comprehension`	[expr for item in iterable] -- builds a list in one line instead of a for-loop with .append() (§0.7).
`f-string (f"...")`	A string literal that evaluates {expressions} inside it and substitutes the result (§0.6).
`raise X from Y`	Exception chaining -- raises a new exception while keeping the original one attached as its cause (§0.8).

ASGI	Asynchronous Server Gateway Interface -- the async-capable successor to WSGI; what Uvicorn implements and FastAPI is built on.
Path operation	A function decorated with @app.get/@app.post/etc. -- FastAPI's term for one route handler.
Depends()	Declares a parameter as resolved by calling another function first -- FastAPI's dependency-injection mechanism.
Pydantic BaseModel	A class whose type-hinted fields double as a validation schema, applied automatically to request bodies.
response_model	Filters/documents a route's return shape -- extra attributes on the returned object are dropped, not leaked.
HTTPException	The way a route (or a dependency) signals a clean, intentional error response.
APIRouter	A mountable group of routes, declared without a reference to the top-level app -- assembled via app.include_router(...).
@lru_cache	Standard-library memoization; on a zero-arg function it acts as a lazy, process-wide singleton.
TestClient / dependency_overrides	FastAPI's in-process testing mechanism: real ASGI app, real routes, swapped dependencies, no live server or credentials.

That's the whole layer: five small files, no reasoning of its own, every route a thin facade over the connector layer and the action graph — built entirely from ordinary FastAPI primitives (routing, Pydantic validation, dependency injection) used consistently enough that the same handful of patterns explain every file.